

搜索，即 test-time compute

肖涵

Elastic 副总裁



微信



模型这边

榜单上挤满了人，第一名和第十名差几个千分点。

换谁家的 embedding，效果都差不多。

链路这边

召回、精排、混合检索、查询改写——该有的组件，十年前就都有了。

于是大家的结论

搜索是个成熟行业，剩下的都是工程活，值得投入的地方在别的方向上。



搜索，本来就是一种
test-time compute。



给你一块 GPU，
你把它花在训练时，
还是推理时？

一句话：**不去训一个更大的模型，而是让手上这个模型，在回答每个问题时多干一点活。**训练阶段的钱已经花完了，这笔新的开销花在推理这一刻。

Best-of-N

采样多个候选答案，选出其中最好的一个。

Self-consistency

让模型把推理过程走多遍，取出现次数最多的答案。

Verifier search

用一个判分器在候选里搜索，投入的算力越多，结果越好。

让机器人在一手扑克上多想 **20 秒**，effect 相当于把模型放大 **10 万倍**、再多训 10 万倍的时间。

Noam Brown, OpenAI, 2024

20 秒，换 10 万倍。

这个兑换比例一旦成立，问题就从「模型够不够大」，变成了「**你舍不舍得在推理时多花这一点**」。



ICML 2026, 韩国首尔, Noam Brown 在讲 test-time compute。



01 | 为什么说搜索就是 TTC

多跑几遍，跑得更深一点，然后看看能换回来什么

向量召回一遍，精排再来一遍，查询改写之后又是一遍，多路结果还要融合。这些环节没有一个是 在训练阶段完成的，**全都发生在用户按下回车之后**。这就是 test-time compute，只是我们一直管它叫「搭链路」。

A

Deeper pass: 把这次前向读透

同一次前向，不只取最后那个池化向量，把中间的 patch、token 全读出来用。**新信息：从这次前向里抄回来的。**

B

More passes: 再跑几遍

同一个冻结模型，换新的视角再编码一次。把文档切成句子，把图片切成 crop。**新信息：从输入里新拿到的。**

C

Calibration: 只变换已有的向量

白化、图传播、最优传输、EM——在**同一批数**上做更花哨的数学。免训练文献大部分在这一类。**新信息：没有。**

三类用的是同一个冻结模型，区别只在**什么东西进了这次计算**。外面还有一层：让 agent 自己把这些串成链路（第 05 章）。

一直以来

推理时间是模型的一个属性

延迟写在 SLA 里，超了就上不了线。它不参与交易——想更准，只能回去换个更大的模型。

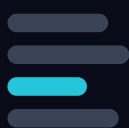
改写成

这个 TALK 里

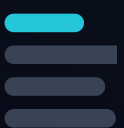
推理时间是一种货币

模型不换、权重不动，只把预算从 10ms 提到 100ms，它能买到什么？

before



after



买到

相关性

同一个任务，排得更准

冻结



买到

新能力

压根没被训练过的事

01

向量模型的 TTC

编码器冻住不动，只在推理期把同一批向量重新组合，相关性就能往上走。

换到的是 · 相关性

02

重排模型的 TTC

召回之后再上一段模型，让候选在同一个上下文里互相比。极端情况下，索引可以整个删掉。

换到的是 · 精度

03

用 TTC 换新能力

同样冻结的模型，这次不求它更准，而是让它做一件训练时根本不存在的任务。

换到的是 · 新能力

04

Agentic search 的 TTC

算力花在 agent 的循环里，由它自己决定先搜什么、用哪个工具、要不要再来一轮。

换到的是 · 召回 + 精度



02 | 向量模型的 test-time compute

编码器冻住不动，只在推理期重新组合它已经算出来的东西

单向量 COSINE



$$\cos(q, d)$$

一篇文档压缩成一个向量

最省算力的做法

句子级 MAXSIM

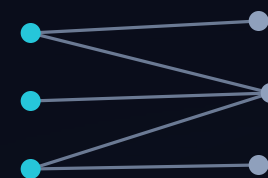


$$\max_s \cos(q, s)$$

还是那个模型，把文档切成句子各算一遍

同一个模型，多算几遍

COLBERT 多向量

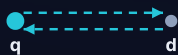


$$\sum_i \max_j \cos(q_i, d_j)$$

每个 query token 对全部 doc token
取 max

需要专门的多向量模型

这十二个程序，没有一个是人设计的



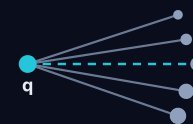
BidirZScore $c=1.2$



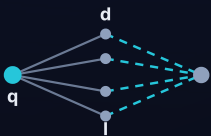
SentMaxSim $c=2.2$



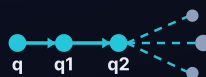
AdaptGranularity $c=2.7$



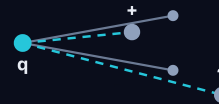
CoverageTriple $c=3.7$



LexicalHybridRRF $c=3.9$



CrossRoundRRF $c=3.9$



DiverseDualCtx $c=5.6$



ConsensusRocchio $c=6.4$



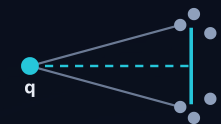
NegContrastive $c=7.2$



MomentumProg $c=9.8$



GraphCentrality $c=12.2$



FisherStability $c=14.7$

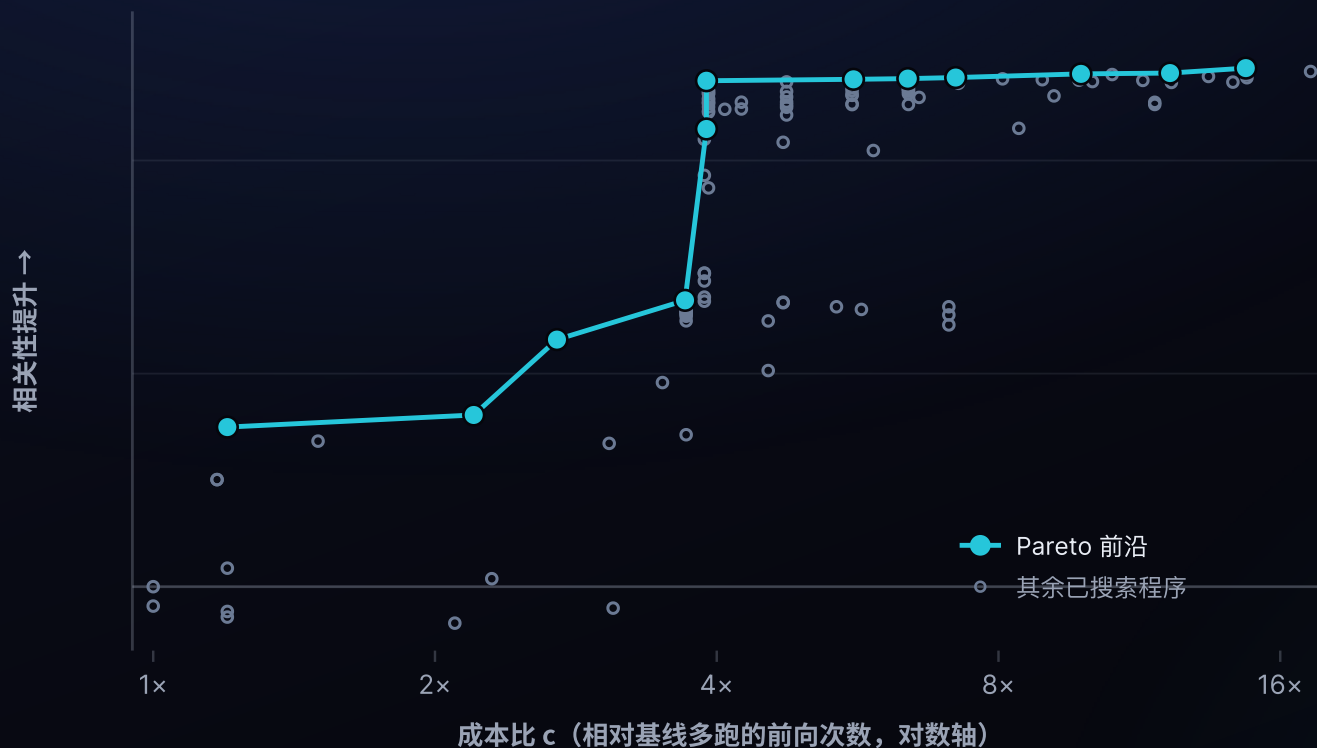


逐格看：换个模型家族，它照样管用



-0.12 +0.16

大多数「任务 × 编码器」组合都拿到了提升。而涨得最好的是右边两块——gemma-300m 和 qwen3-0.6b 完全没参与过发现。



144

搜出来的程序总数，全部是对同一批冻结向量的免训练重组

12

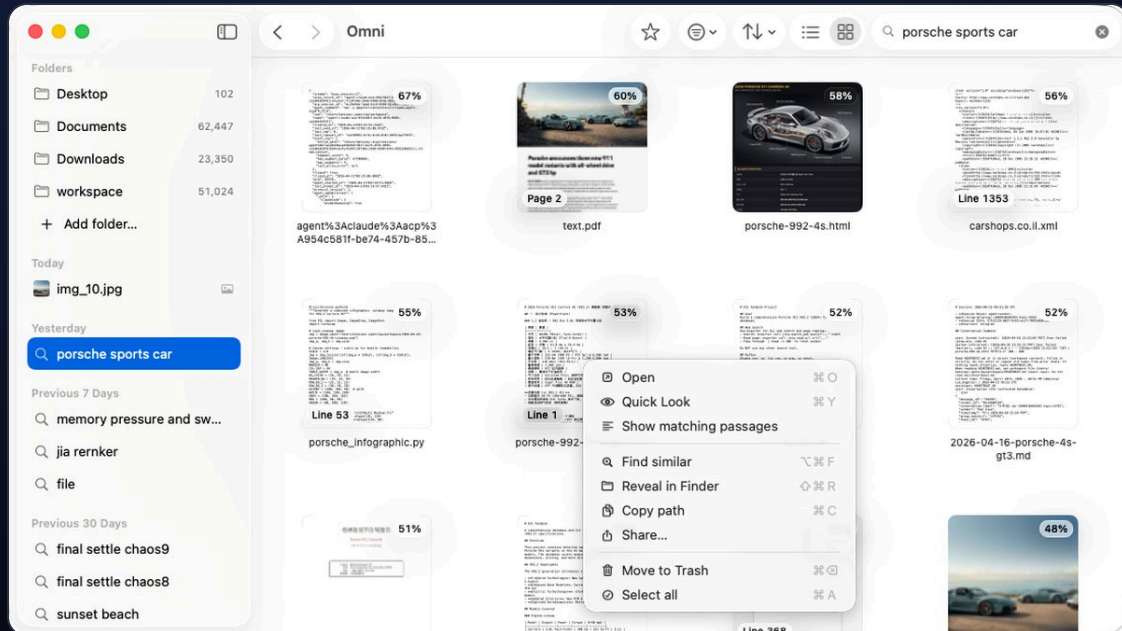
落在 Pareto 前沿上，从最便宜的一档排到最贵的一档

这就是一条标准的 **test-time compute scaling 曲线**，而它发生在一个冻结的小模型上：越往右算得越多，精度就越往上走。



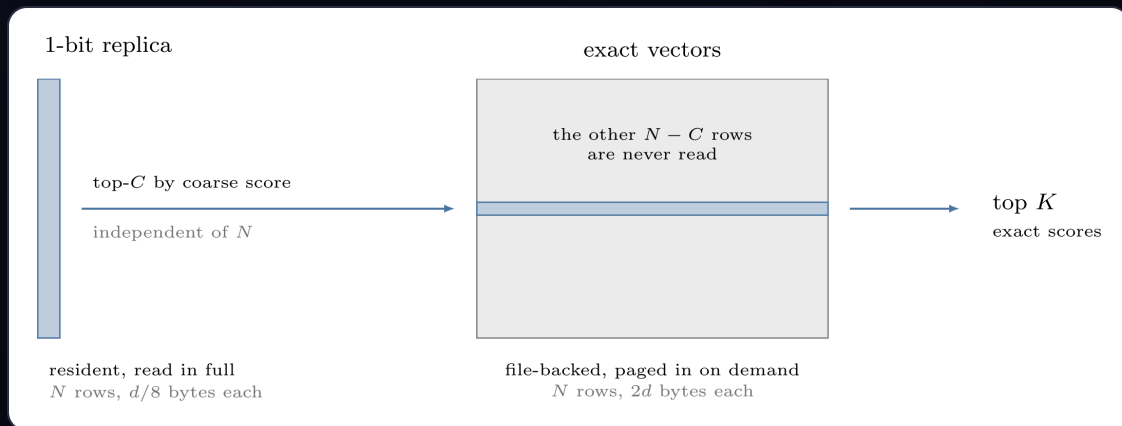
03 | 重排：最直接的 test-time scaling

召回给你一堆候选，重排再花一次算力，把它们放在一起比一遍



Omni 里没有 ANN 索引，一次查询要扫全库。建一个图索引，光链接就要每行 256 字节，四百万 chunk 时是 0.95 GB，**接近查询要扫的矩阵的三倍**，而且持续编辑的语料会不断让它失效。

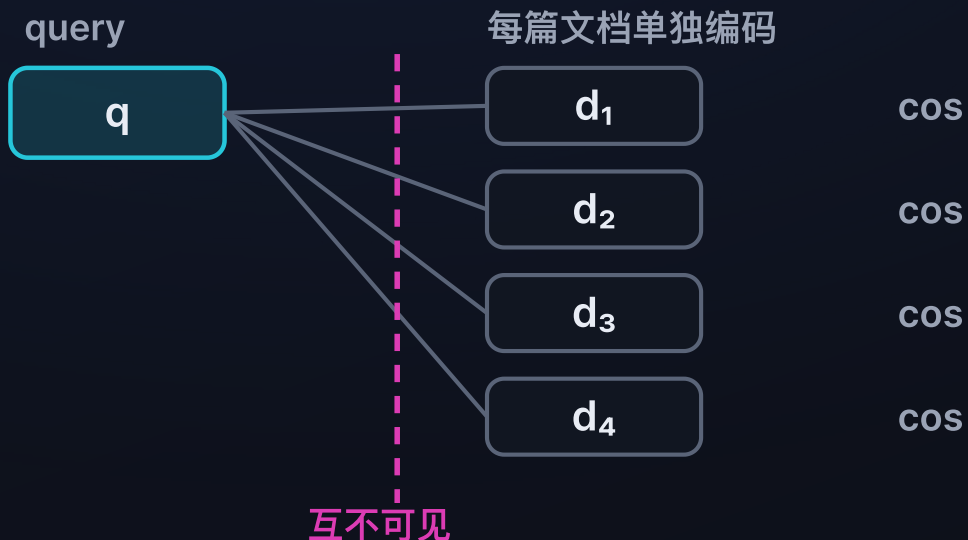
所以同一个空间存两份：**扫描读 1 bit 副本**，**精确向量只有短名单才碰**。粗排分数只决定谁进短名单，不必准，只要短名单够宽；**最终排序由精确重算说了算**。候选集是 $\min(4096, \max(1024, \text{topK} \times 32))$ 。



副本只有精确向量的十六分之一宽；青色是一次查询真正读到的部分。

双塔 bi-encoder

召回



每篇文档的向量，是在别的文档还不存在的时候就算好的。四篇各打各的分，谁也没见过谁。便宜，能扫全库。

listwise 重排

精排



query 和最多 64 篇候选放进同一个上下文，彼此可见。三段都提到延迟时，模型能判断哪一段真的回答了问题。

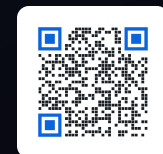
同样 0.6B 参数，listwise 的 jina-reranker-v3 明显甩开双塔式的 bge-reranker-v2-m3 和 Qwen3-Reranker-0.6B。差距来自让候选互相可见，不是来自更大的模型。



只有一页文档时，把索引删掉



GITHUB



常规做法是把每个 chunk 都 embed、存向量、再用 ANN 查回来。但索引是靠很多次查询摊薄成本的，而一页文档只活在这一次浏览里。为它建索引，比直接跑一遍前向还贵。

所以这里反过来：**把整页塞进一次 listwise 前向**，答案直接以 <mark> 标回原文，页面自己的 CSS、表格、图都不动。

一百万篇文档仍然需要第一段召回。而**只有一页的时候，该删掉的正是召回这一段。**

in-page qa
one listwise jina-reranker-v3.5 call ranks every passage of a page. no index, no vector store.

source

page url
https://jina.ai/news/jina-reranker-v3.5

question
how did they make it run faster without making the model bigger

ranking

granularity: 2 sentences, highlights: top 1

search

result

sentences	104
chunks	52
latency	271 ms
requests	1

1. 0.3136
The practical class of jina-reranker-v3.5 is narrow and testable: at 0.6B parameters, targeted training closes most of the gap to a 4B generalist reranker, and on BEIR it closes all of it...
Highlights are rationed into the original html, so the page keeps its own stylesheet and layout. model runs locally through aio.

https://jina.ai/news/jina-reranker-v3.5-faster-listwise-reranking-hybrid-attention-self-distillation/

elastic

jina

```
green tea contains catechins that may help reduce inflammation. ,
"El precio del café ha aumentado un 20% este año.",
"绿茶富含儿茶素等抗氧化剂，可以降低心脏病风险。",
"Le thé vert est riche en antioxydants.",
]
```

```
for r in model.rerank(query, documents):
    print(f"{r['relevance_score']:.4f} {r['document']}[:70]}")
```

Conclusion

The practical claim of `jina-reranker-v3.5` is narrow and testable: at 0.6B parameters, targeted training closes most of the gap to a 4B generalist reranker, and on BEIR it closes all of it, while running faster than the model it replaces. For enterprise retrieval that argues for investing in focused supervision on a compact backbone rather than defaulting to the largest model available.

Two limitations are worth stating plainly. Listwise rerankers still carry input constraints that pointwise and late-interaction models avoid, in particular fixed upper bounds on candidate count and total candidate length. And important gaps to the 4B Qwen remain on RTEB legal and medical tasks, controlled-pool Struct-IR, and low-resource MIRACL languages. Those are the hard cases, and we name them rather than hide them behind an average.

Categories: Featured Press release

问题里没有一个实词出现在它找到的那句话里。

search_web 引擎自带 snippet

 Tencent Cloud
<https://intl.cloud.tencent.com/product>

Elasticsearch Service

Tencent Cloud Elasticsearch Service (ES) is a fully managed, elastically scalable cloud-native service with enterprise-grade high availability, ... [Read more](#)

 Tencent Cloud
<https://www.tencentcloud.com/products/tem>

Tencent Cloud Elastic Microservice

Tencent Cloud Elastic Microservice (TEM) is a serverless PaaS platform designed for microservice applications. It perfectly combines serverless resources and ... [Read more](#)

 Tencent Cloud
<https://intl.cloud.tencent.com/product/emr>

Elastic MapReduce

Tencent Cloud Elastic MapReduce (EMR) provides secure and cost-effective cloud-based Hadoop services featuring high reliability and elastic scalability.

 Elastic — The Search AI Company
<https://www.elastic.co/blog/elastic-tencent-cloud-ai-s...>

Elastic and Tencent Cloud AI search technology make ...

Jul 14, 2025 — The Tencent Cloud Elasticsearch Service is a one-stop AI search and log analysis service on the cloud.

 Gartner

每条下面那行就是引擎选的 snippet。

search_web_deep meta=deep

01 拿 SERP 候选

read_num=20

TEST-TIME COMPUTE

02 Reader 并发读正文

按需派发 · 没有 barrier

03 句子边界切块

100 词 · 重叠 25

04 所有页的所有块 + 引擎 snippet

入同一个候选池

05 一次 listwise 打分

jina-reranker-v3.5

06 每个 url 取最高分那一块

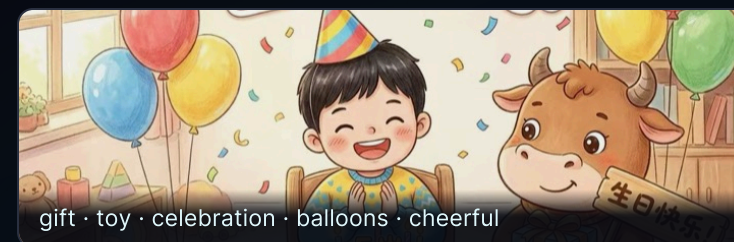
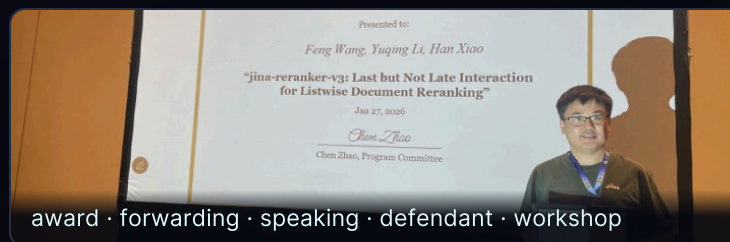
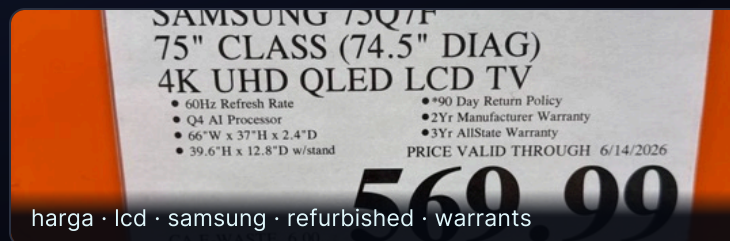
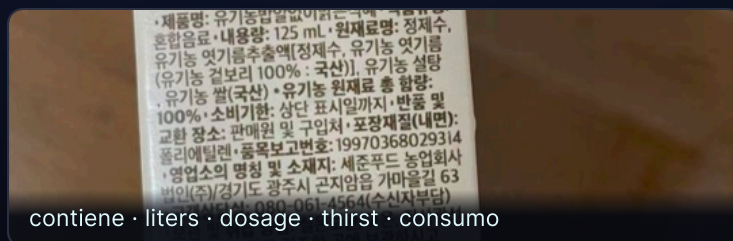
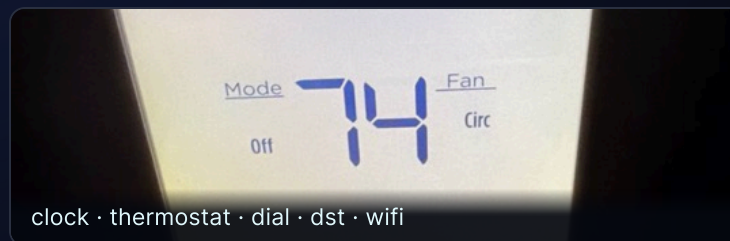
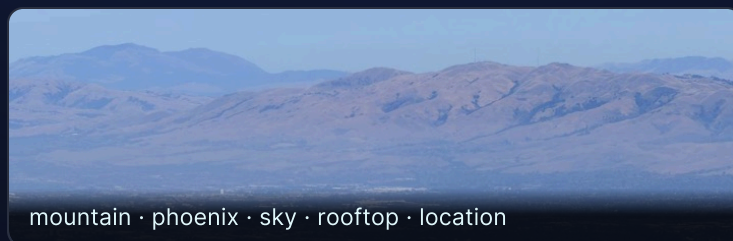
跨页同尺度，不需阈值



04 | 用 test-time compute 换一个新能力

一个只会做检索的模型，从来没学过打标签。这次不求它更准，求它会一件新事

这些标签出自一个检索模型



没训练、没请第二个模型、没查词典，全部由推理期的计算得到，而且天然是多语言的。

- | 手上有 一个 **jina-embeddings-v5-omni-nano**，大约 1B，图和文共用一个 768 维空间。
- | 要它干 把图里**每一个**东西都说出来：多标签，而且词表是开放的。
- | 前提是 权重全冻结。它是个检索编码器，**没有标注头，没有分类器，训练时压根没见过这个任务。**
- | 只准用 它自己吐出来的那些数，在上面做推理期计算。别的一概不行。

不训练

不请第二个模型

不用 WordNet 和词典

不用正则和词性标注

A

Deeper pass: 同一次前向，读得更深

别只拿那个池化完的向量，把 **patch 级** 的输出全读出来，挨个跟 12.8 万个词打分。

新信息：从这次前向里抄回来

B

More passes: 换个视角，再跑一遍

把图切成若干 crop **重新编码**。小物体在自己那张 crop 里占满画面，不再被整图的池化冲淡。

新信息：从输入里新拿到

C

Calibration: 在已有结果上做文章

白化、最优传输、图传播、EM——在**同一批像素**上做更花哨的数学。

新信息：没有

A 换来一大截，**B** 再补一小截，**C** 一样都没换来。

离线，只算一次



每张图





$$S(\ell) = \underbrace{0.3 \left(\langle g, e_\ell \rangle - \mu_\ell^g \right)}_{\text{全局上下文}} + \underbrace{0.7 \left(\max_{p \in P} \langle p, e_\ell \rangle - \mu_\ell \right)}_{\text{patch 证据}} + \underbrace{1.3 \left(\max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell \right)}_{\text{多 crop 重编码}}$$

全局上下文：池化向量减掉自己的背景先验。不额外花钱。

A patch 证据 · Deeper pass：在前向已经产生的那些行上做代数，不额外花钱。

B 多 crop · More passes：14 次新的前向，看新的像素。

然后 $\ell \in$ 词首过滤后的词表 $\rightarrow$ 向量空间 $\text{NMS}(\tau = 0.6) \rightarrow \text{top-}k$

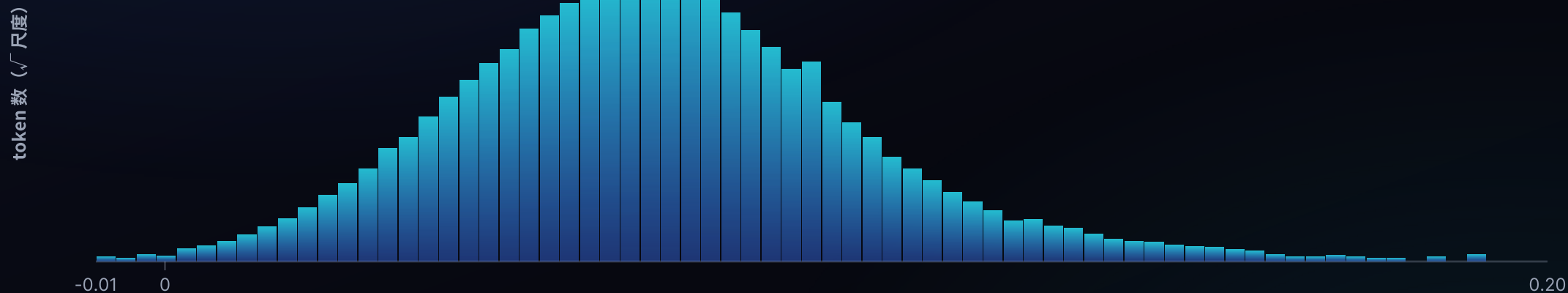
没有分类头，没有 logits，没有学出来的阈值：三个固定权重、两个背景先验，以及若干次 max。



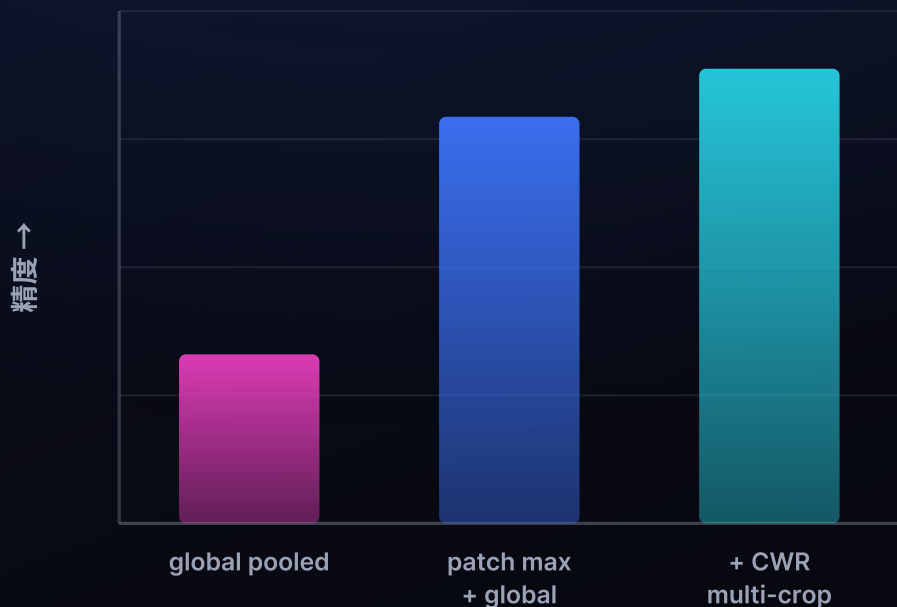
12.8 万个词的得分如何被整理出来

1· 原始融合得分

全部 128,260 个 token · 0.7 patch-max + 0.3 global



一条光滑的钟形：每个词都跟每张图「有点像」



只用池化向量



A 改读 patch

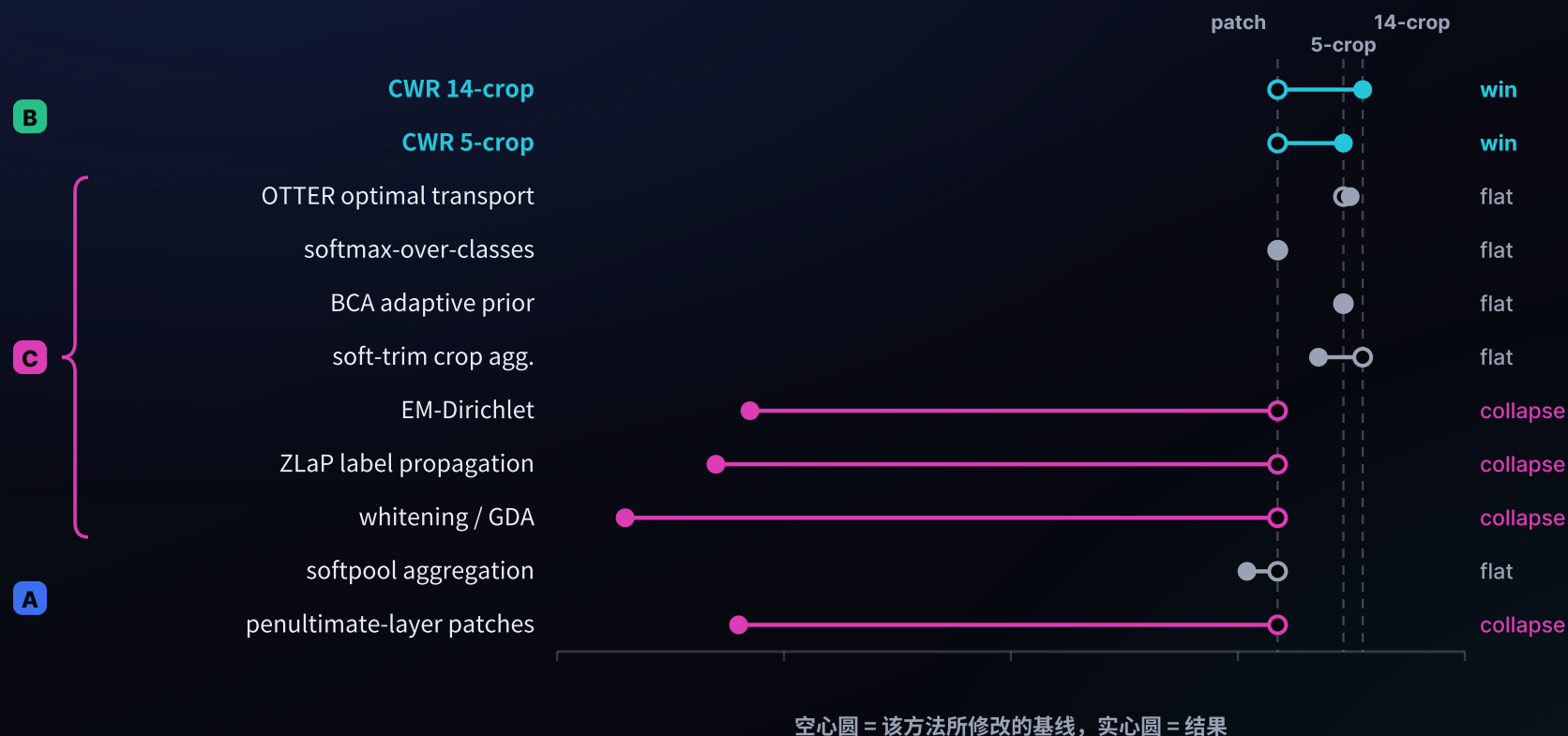


B 再切几个 crop



一步跨得最大的是 **A**：把这次前向已经算出来的 patch 读完整。**B** 再去看新的像素，还能再补一段。

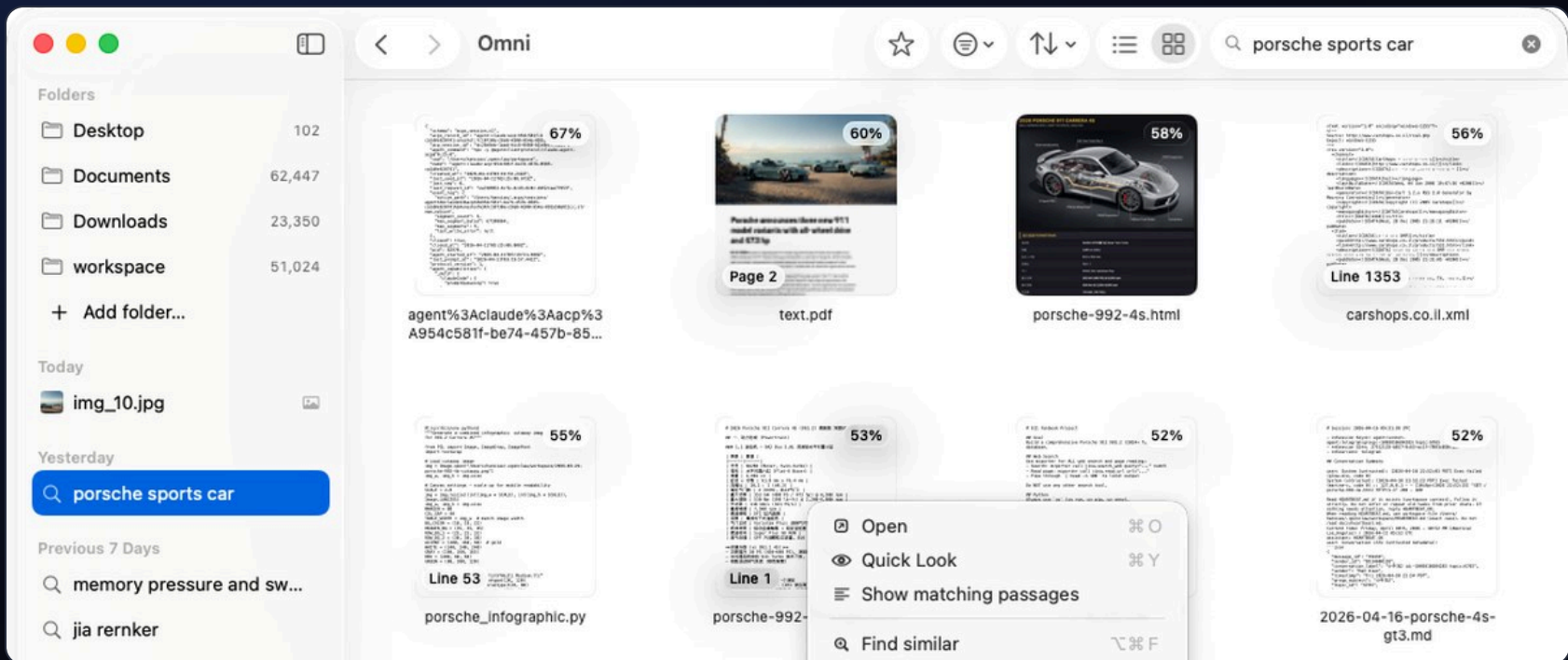
这中间模型一个权重都没动过。



能站住的只有 B。C 那些数学假定图和文在两个空间里，可这儿本来就是一个空间。



前沿上的每一步，都让模型读到了上一步没读到的信息。



图片

建索引时一并产出标签，不需要第二次前向。

视频

抽帧后共用同一套 patch 打分逻辑，时间维并入同一次 max。

扫描版 PDF

当图片处理。OCR 无法识别的扫描件，也能用词检索到。

全跑在那台存着文件的 Mac 上。从实验记录到能用的功能，中间只隔着一点点额外算力。



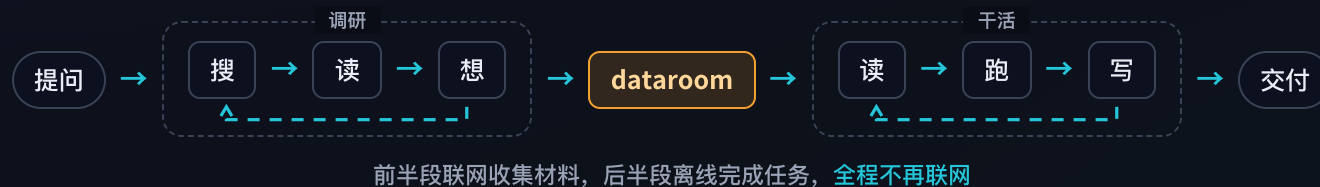
05 | Agentic search 里的 test-time compute

前面两个项目都在模型内部。这一层，算力花在了 agent 的循环里

2025 · Deep research



2026 · 长程任务



调研和执行需要的不是同一种 token。收集材料用便宜的本地模型，昂贵的额度留给真正需要判断力的那一段。

dataroom: 把互联网下成一个 zip



GITHUB



给它一个题目和一个 token 预算，它**搜、读、写**循环推进，把互联网上跟这个题目相关的内容**下成一个带引用的 zip**。

关键是 token 的账：探索阶段用自己部署的 Qwen3.6-35B-A3B、Qwen3.8-27B 跑，把昂贵的前沿额度省下来，留给后面真正需要判断力的那一步。

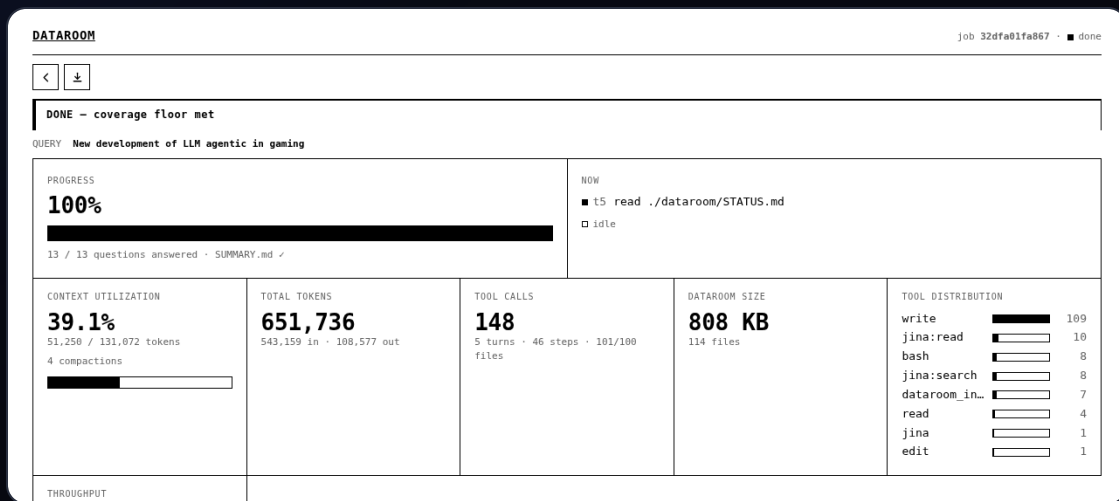
dataroom

下成语料 (.zip)



searchbox

只从这个 zip 里找答案



停止条件是覆盖度达标，不是预算耗尽。

Unlock the Power of Search AI

AI 搜索技术大会

searchbox: 把 agent 关进没有网的盒子

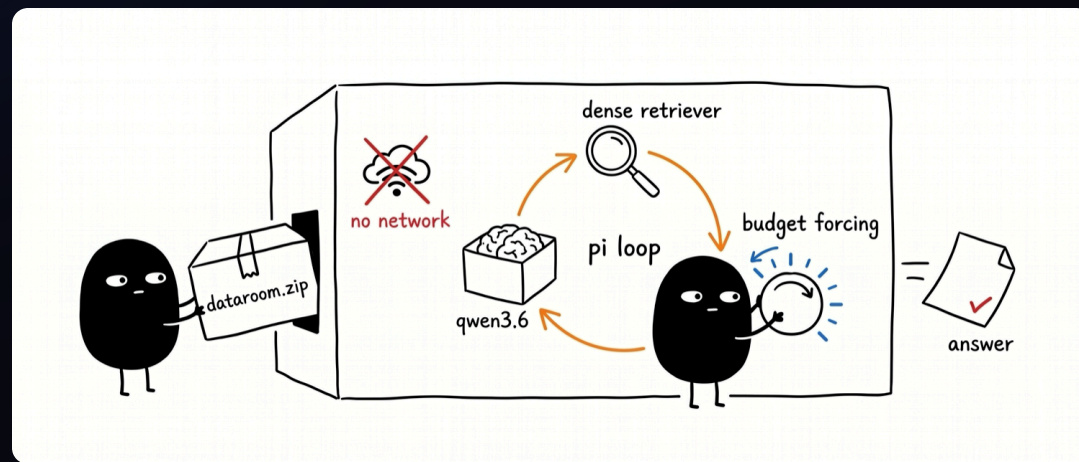


给它一个 dataroom 的 zip，**切断网络**，再问一个问题。自己部署的 Qwen3.6-35B-A3B 在里面跑。它想答上来，只能**用手边的本地工具现搭一条链路**——grep、embed、rerank、相似度、聚类、选多样性。

断网这件事是故意的。**不断网，就分不清它是真的在检索，还是网上碰巧有现成答案。**

正在研究的几个问题

- 它最先调用的是哪个工具？
- 是不是 grep 就够了，稠密检索到底在哪些地方才真有用？
- 提高算力预算，难题上的正确率会不会上升？
- 它搭出来的链路，下次还能不能复用？



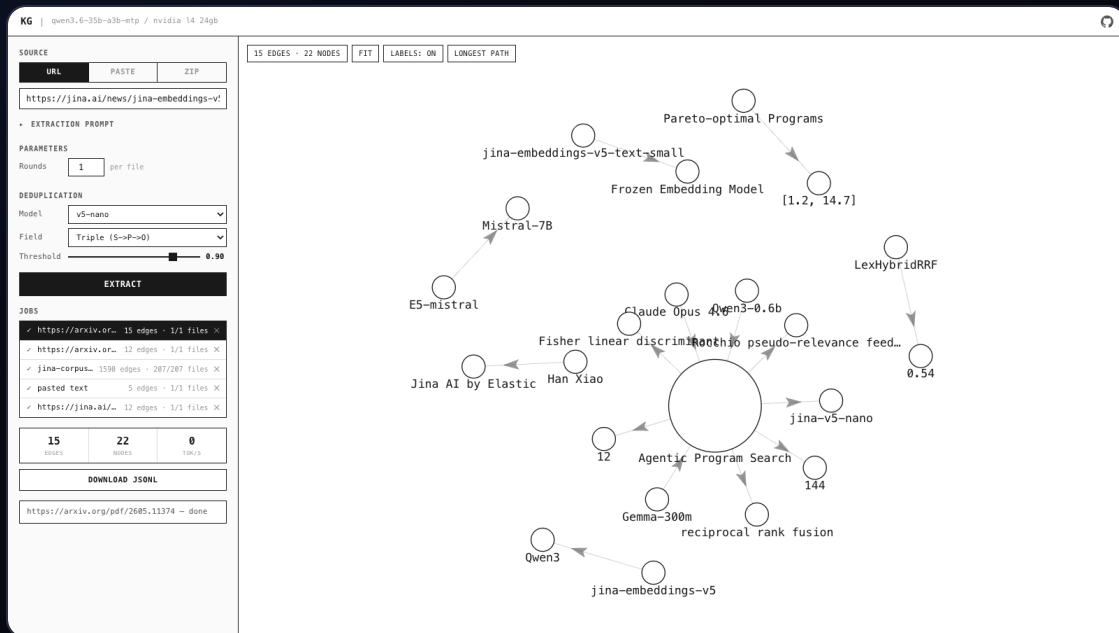


简单题对研究 agent 搜索毫无用处：**一次 grep 就能命中的问题，什么方法做出来都一样分。**要评测 searchbox，得先有真正需要搜索才能答的题。

怎么造

把语料先抽成知识图谱，每条事实是一条 (主语)-[谓语]->(宾语) 的边，然后去**走图里最长的那些路径**。这些长链条自然就成了**多跳问题**——没有任何单独一段文字能直接回答。

题目从同一份语料里长出来，所以是**私有的、不可能被背过的评测集**。agent 必须真的把事实一环一环连起来，也就必须真的去花那份推理算力。



每条事实一条边，最长路径就是多跳题。

前面三个是完整的链路。想知道**检索这一环到底值多少**，得把它单独拎出来：固定一份 Jina 文档语料（210 篇 md 加结构化 json，切成 7777 个 chunk），外面套一个 agent，然后**只给它一个外部工具**。

其余全是 Pi 自带的 read、grep、bash。这样被测的就只有 search_corpus 这一件事，**结果的好坏只能归因于它**。

命中不是答案

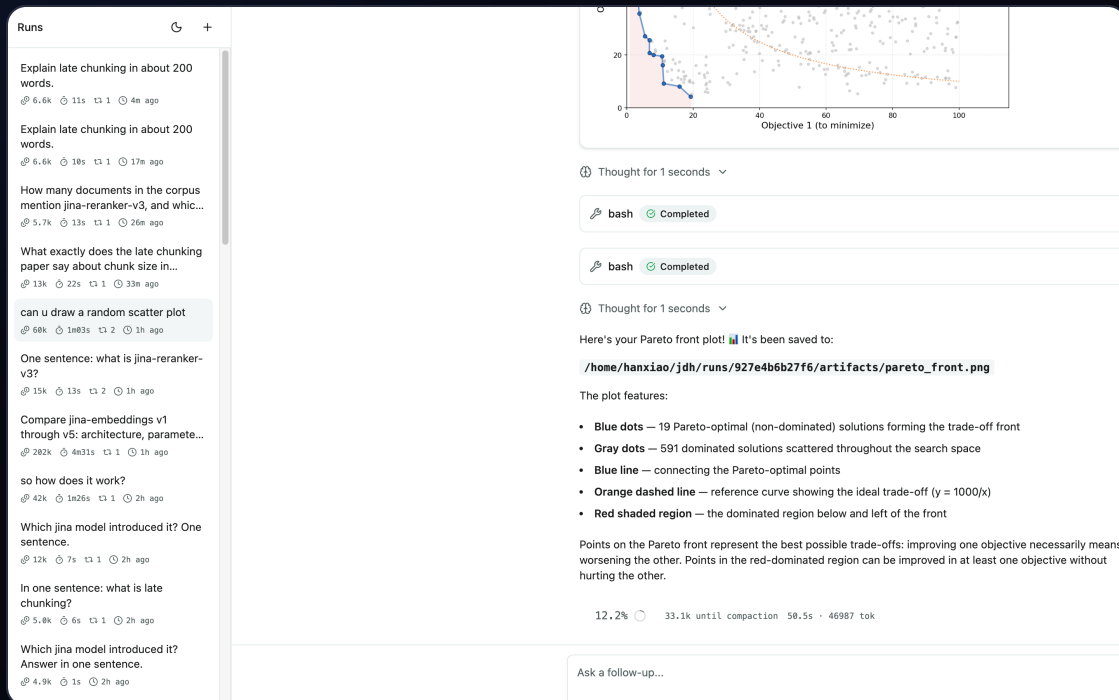
返回 {score, path, lines, text}，agent 得自己回原文把段落读完、再 grep 一遍看还有哪儿提到。

没有 SIDECAR

7777×768 的矩阵直接在扩展里算：加载 117ms，单次检索 4.3ms。少一个需要启动、占用端口、可能失效的服务。

联网是开关

默认关。开了才挂上 search_web 和 read_url，于是「翻书」和「上网」可以分开计分。



思考流、工具调用、上下文占用、每段耗时，全部流式开着。

01

dataroom · 把召回做满

便宜的本地 token 把互联网下成一个带引用的 zip。这一步决定了天花板——材料里没有的东西，后面谁也变不出来。

02

searchbox · 把精度做满

断网，只给本地工具，让 agent 在推理期自己组装链路。这一步决定了能从材料里取出多少。

03

知识图谱 · 评测基准

从同一份语料生成多跳难题。前面两步的每一次改动，都靠它来衡量。

04

harness · 单独衡量检索

固定语料，只留一个检索工具。把链路里那一环的贡献单拎出来看。



06 | 回到那句话

四块地方，同一笔账

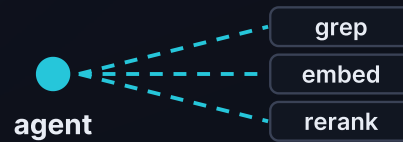


一层在模型里面，一层在模型外面



A 模型内：多算几遍

推理时干的	重组已有向量，加一段模型，或把这次前向读透
项目	向量代数重组 · 重排 · 页内检索 · 图像标注
买到的	精度 + 一个全新的能力



B 模型外：工具链

推理时干的	自己决定先搜什么、再用什么
项目	dataroom · searchbox · 知识图谱 · harness
买到的	召回 + 精度

两层，一个赌注：多花推理算力，而不是换个更大的模型。

01 2026 不是做搜索最好的一年，也不是最差的一年。

最好是 2025，deep search / deep research 兴起，一提大模型落地就是它俩。最差是 2024，所有人都去做 RAG，觉得传统检索和 reranker 没用了。2026 注意力转向 long-horizon task，搜索的比重被稀释了——但大模型不盯着这块，反而给我们留出了发挥的空间。

02 test-time compute 是一种思维，不是一项技术。

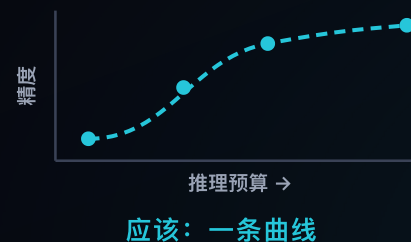
遇到做不好的事情，先别急着去凑数据、去换更大的模型。先问一句：这件事能不能在推理时解决。

03 test-time scaling 不等于 training-free。

两件事是正交的：一个说推理时花多少算力，一个说权重动不动。最强的 TTC 恰恰是训出来的——listwise 重排得先把模型训成能在一个 context 里比候选，long thinking 得先用 RL 训出来。冻结模型只是最省的那种做法，不是定义。

04 今天的搜索模型和 benchmark，几乎都没把 test-time compute 算进去。

榜单上一个模型就一个分数，默认大家都只允许算一遍。应该报的是一条曲线：横轴推理预算，纵轴精度，看它在这个平面上往哪儿爬。一个点测不出 test-time scaling。

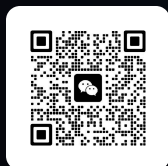




搜索，就是 **test-time compute**。
高质量搜索，就是 **test-time scaling**。

肖涵 · Elastic 副总裁

微信



OMNI



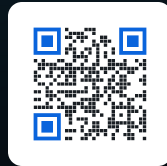
IN-PAGE SEARCH



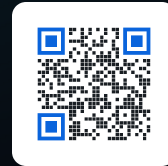
IMAGE TAGGING



DATAROOM



SEARCHBOX



KNOWLEDGE-GRAPH

